this book also records and displays the values from previous requests so you can easily track how the values change with each request.

Listing 9.12 The `RowCounts` handler depends on `DataContext` and `Repository`

```
static string RowCounts(                                                    ❶
    DataContext db,                                                         ❶
    Repository repository)                                                  ❶
{
    int dbCount = db.RowCount;                                              ❷
    int repositoryCount = repository.RowCount;                              ❷

    return: $"DataContext: {dbCount}, Repository: {repositoryCount}";       ❸
}
```

❶ DataContext and Repository are created using DI.

❷ When invoked, the page handler retrieves and records RowCount from both dependencies.

❸ The counts are returned in the response.

The purpose of this example is to explore the relationships among the four `DataContext` instances, depending on the lifetimes you use to register the services with the container. I'm generating a random number in `DataContext` as a way of uniquely identifying a `DataContext` instance, but you can think of this example as being a point-in-time snapshot of, say, the number of users logged on to your site or the amount of stock in a warehouse.

I'll start with the shortest-lived lifetime (transient), move on to the common scoped lifetime, and then take a look at singletons. Finally, I'll show an important trap you should be on the lookout for when registering services in your own apps.

9.4.1 Transient: Everyone is unique